



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 5

Planificación de caminos II

Algoritmo A con heurística externa y análisis de admisibilidad*

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: MATLAB

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	3
2.1. Algoritmo A*	3
2.2. Admisibilidad de la heurística	4
2.3. Reconstrucción de la ruta	4
3. Diseño e implementación	4
3.1. Interfaz de la función	4
3.2. Estructuras del algoritmo	4
3.3. Bucle principal	5
3.4. Validaciones de entrada	5
4. Resultados experimentales	5
4.1. Datos del enunciado	5
4.2. Casos de validación	6
5. Discusión	7
5.1. Por qué falla el caso $7 \rightarrow 4$	7
5.2. Heurística admisible propuesta	7
5.3. Limitaciones	8
6. Conclusiones	8

1. Introducción

1.1. Objetivo de la práctica

Implementar el algoritmo A^* en MATLAB como una función que reciba como parámetros la matriz de costes G , una matriz de heurísticas H pasada externamente, un nodo origen y un nodo destino. Además del algoritmo en sí, la práctica plantea ilustrar el papel crítico que juega la **admisibilidad** de la heurística: cuando H sobreestima el coste real, A^* puede devolver rutas **subóptimas**.

1.2. Descripción del problema

Dado un grafo dirigido $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ con N nodos y matriz de costes $G \in \mathbb{R}^{N \times N}$ ($G(i, j) > 0$ representa el arco $i \rightarrow j$ con peso $G(i, j)$), una matriz de heurísticas $H \in \mathbb{R}^{N \times N}$ ($H(i, j)$ es la estimación del coste de i a j), y dos nodos s (origen) y t (destino), se desea encontrar:

1. el coste de la ruta hallada por A^* desde s hasta t ,
2. la secuencia de nodos $\pi = [s, \dots, t]$ que la define.

La firma exacta de la función es:

```
[coste, ruta] = aestrella(G, H, origen, destino)
```

2. Fundamentos teóricos

2.1. Algoritmo A^*

A^* es un método de **búsqueda informada** en grafos que en cada paso expande el nodo abierto con menor valor de la función de evaluación

$$f(n) = g(n) + h(n), \quad (1)$$

donde $g(n)$ es el coste real acumulado desde el origen hasta n y $h(n) = H(n, t)$ es la estimación heurística del coste restante hasta el destino t .

Dijkstra (Lab-MR 4) es el caso particular $h \equiv 0$, en el que A^* degenera en búsqueda desinformada uniforme. Cualquier $h(n) > 0$ que sea *informativo* reduce el número de nodos explorados respecto a Dijkstra.

2.2. Admisibilidad de la heurística

Sea $h^*(n)$ el coste real mínimo desde n hasta el destino. Una heurística h es **admisble** si

$$h(n) \leq h^*(n) \quad \forall n, \quad (2)$$

es decir, **nunca sobreestima** el coste real. La admisibilidad es condición suficiente para que A^* devuelva la ruta óptima.

Cuando h sobreestima en algún nodo, A^* puede expandir prematuramente el destino al alcanzarlo por una ruta de mayor coste, descartando rutas **más baratas pero todavía no exploradas** cuyas estimaciones f son artificialmente altas. El resultado es una ruta válida pero **no óptima**.

2.3. Reconstrucción de la ruta

Como en Dijkstra, se mantiene un vector de predecesores $\pi[v]$ que se actualiza al relajar aristas. Al terminar el bucle, se reconstruye la ruta retrocediendo desde t hasta s .

3. Diseño e implementación

3.1. Interfaz de la función

La función se ha implementado como `aestrella.m` con la signatura del enunciado:

```
[coste, ruta] = aestrella(G, H, origen, destino)
```

Importante: la heurística se pasa **como matriz H externa**, no se calcula internamente a partir de coordenadas u otros datos geométricos. Esto sigue estrictamente el enunciado y permite estudiar el efecto de heurísticas no admisibles sin modificar la función.

3.2. Estructuras del algoritmo

Se mantienen vectores de tamaño N :

- $g_score(v)$: mejor coste real conocido de s a v . Inicialmente ∞ , salvo $g_score(origen) = 0$.
- $f_score(v) = g_score(v) + H(v, destino)$: criterio de expansión.

- $\text{prev}(v)$: predecesor en la ruta óptima parcial.
- $\text{abierto}(v)$, $\text{cerrado}(v)$: máscaras booleanas.

3.3. Bucle principal

1. Selecciona el nodo abierto con menor f .
2. Si es el destino, corte temprano y reconstrucción.
3. En caso contrario, lo cierra y relaja sus aristas salientes:

$$\text{si } g(u) + G(u, v) < g(v) \implies g(v) \leftarrow g(u) + G(u, v), f(v) \leftarrow g(v) + H(v, t), \pi(v) \leftarrow u \quad (3)$$

3.4. Validaciones de entrada

Antes del bucle se comprueban: G y H cuadradas $N \times N$ del mismo tamaño, costes y heurísticas finitos y no negativos, origen y destino en rango, y caso trivial $\text{origen} == \text{destino}$.

4. Resultados experimentales

4.1. Datos del enunciado

El enunciado proporciona un grafo G de 7 nodos junto con una matriz de heurísticas H :

$$G = \begin{bmatrix} 0 & 2 & 0 & 0 & 9 & 0 & 0 \\ 2 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 5 & 0 & 0 & 0 \\ 0 & 2 & 5 & 0 & 1 & 2 & 4 \\ 9 & 0 & 0 & 1 & 0 & 4 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 1 \\ 0 & 0 & 0 & 4 & 0 & 1 & 0 \end{bmatrix} \quad H = \begin{bmatrix} 0 & 2 & 5 & 5 & 5 & 6 & 9 \\ 2 & 0 & 2 & 3 & 6 & 6 & 8 \\ 5 & 2 & 0 & 2 & 6 & 6 & 7 \\ 5 & 3 & 2 & 0 & 5 & 4 & 6 \\ 5 & 6 & 6 & 5 & 0 & 1 & 5 \\ 6 & 6 & 6 & 4 & 1 & 0 & 1 \\ 9 & 8 & 7 & 6 & 5 & 1 & 0 \end{bmatrix}$$

4.2. Casos de validación

Se han ejecutado los tres casos planteados por el enunciado. Los resultados se resumen en la Tabla 1 y se ilustran en la Figura 1.

Tabla 1: Casos de validación del algoritmo A* sobre el grafo del enunciado.

Caso	Coste	Ruta	Observación
<code>aestrella(G, H, 1, 7)</code>	7	[1 2 4 6 7]	Óptimo. La H del enunciado es admisible para los nodos relevantes en este recorrido.
<code>aestrella(G, H, 7, 4)</code>	4	[7 4]	Subóptimo ($h^*(7, 4) = 3$ por la ruta $7 \rightarrow 6 \rightarrow 4$). $H(7, 4) = 6$ sobreestima, A* expande 4 directamente desde 7 y termina.
<code>aestrella(G, H', 7, 4)</code>	3	[7 6 4]	Óptimo . Con H' admisible (ver §5.2) A* recupera la ruta de menor coste.

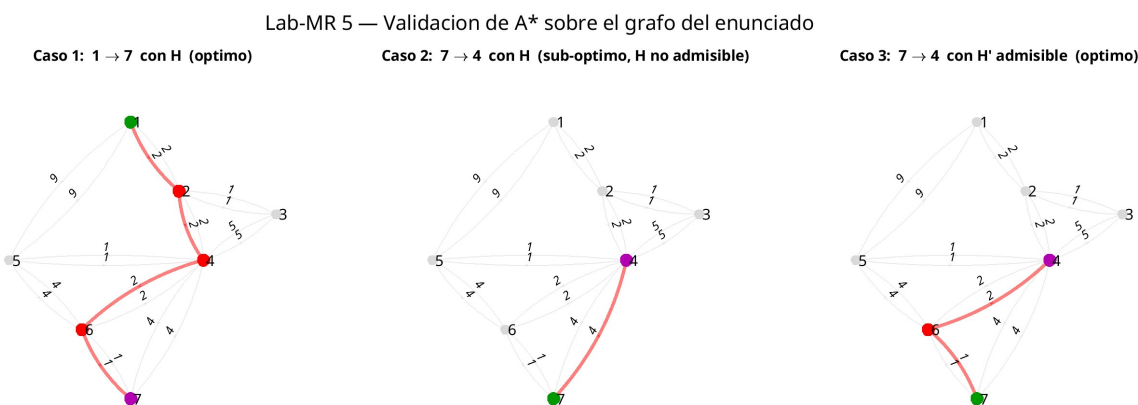


Figura 1: Tres casos de validación sobre el grafo del enunciado. Verde: origen; magenta: destino; rojo: ruta calculada por A*. (Izquierda) $1 \rightarrow 7$ con H original — óptimo. (Centro) $7 \rightarrow 4$ con H original — la ruta $[7, 4]$ tiene coste 4 pero la óptima es $[7, 6, 4]$ con coste 3; A* falla por sobreestimación. (Derecha) $7 \rightarrow 4$ con H' admisible — A* recupera el óptimo.

5. Discusión

5.1. Por qué falla el caso $7 \rightarrow 4$

Trazando el bucle de A^* paso a paso con la H original:

1. Se inicia en 7 con $g(7) = 0$, $f(7) = H(7, 4) = 6$.
2. Se expande 7. Sus vecinos en G son 4 (peso 4) y 6 (peso 1):
 - 4: $g(4) = 4$, $f(4) = 4 + H(4, 4) = 4 + 0 = 4$.
 - 6: $g(6) = 1$, $f(6) = 1 + H(6, 4) = 1 + 4 = 5$.
3. El nodo abierto con menor f es 4 (justo el destino). A^* aplica corte temprano: $\text{coste} = 4$, $\text{ruta} = [7, 4]$.

A^* nunca llega a explorar 6, aunque la ruta $7 \rightarrow 6 \rightarrow 4$ tiene coste real 3 (menor que 4). El motivo es que $H(6, 4) = 4$ sobreestima el coste real $h^*(6, 4) = 2$, lo que infla artificialmente $f(6)$ y empuja a 6 por debajo de 4 en la cola de prioridad.

5.2. Heurística admisible propuesta

Una heurística admisible para el destino 4 debe satisfacer $H'(i, 4) \leq h^*(i, 4)$ para todo i . Calculando $h^*(\cdot, 4)$ con Dijkstra sobre G se obtienen los costes reales mínimos a 4:

$$h^*(\cdot, 4) = [4 \ 2 \ 3 \ 0 \ 1 \ 2 \ 3]^\top$$

Sustituyendo la columna 4 de H por estos valores se obtiene la matriz H' . Esta es la heurística **perfectamente informada** (igualdad $H' = h^*$), y trivialmente cumple la condición de admisibilidad. Con H' , el bucle de A^* explora $7 \rightarrow 6$ antes que $7 \rightarrow 4$ porque ahora $f(6) = 1 + 2 = 3 < f(4) = 4$, y desde 6 encuentra la ruta óptima al destino.

Existen alternativas menos potentes pero igualmente admisibles, por ejemplo:

- $H'_{\text{cero}}(i, j) = 0$: trivialmente admisible, pero hace que A^* degenere en Dijkstra (sin ganancia computacional).
- $H'_{\text{BFS}}(i, j) = c_{\text{mín}} \cdot d_{\text{aristas}}(i, j)$, donde $c_{\text{mín}}$ es el coste mínimo de cualquier arista del grafo y d_{aristas} es la distancia en número de aristas. Es admisible y más informativa que cero.

La elección entre estas opciones es un compromiso entre coste de cómputo de la heurística y calidad de la búsqueda. Para grafos pequeños como el del enunciado,

h^* exacta calculada con Dijkstra es factible y óptima.

5.3. Limitaciones

- Una heurística admisible garantiza optimalidad pero no necesariamente es **consistente** (monótona). Para implementaciones más sencillas con búsqueda en grafos (sin reapertura de nodos cerrados) la consistencia es preferible.
- La búsqueda lineal del mínimo en cada iteración da complejidad $O(N^2)$. Para grafos grandes sería más eficiente una cola de prioridad.
- La función no contempla heurísticas dependientes del estado más allá de la matriz H pasada (suficiente para el enunciado).

6. Conclusiones

Se ha implementado A^* sobre grafos dirigidos con la signatura exacta del enunciado, recibiendo G , H , origen y destino, y devolviendo coste y ruta. La validación con los tres casos del enunciado confirma:

1. Para $1 \rightarrow 7$ con la H original, A^* recupera la ruta óptima $[1, 2, 4, 6, 7]$ con coste 7.
2. Para $7 \rightarrow 4$ con la misma H , A^* devuelve $[7, 4]$ con coste 4, **peor que el óptimo** $[7, 6, 4]$ de coste 3. La causa es la sobreestimación $H(7, 4) = 6 > h^*(7, 4) = 3$, que hace que H no sea admisible.
3. Sustituyendo la columna 4 de H por los valores reales $h^*(\cdot, 4)$, la heurística resulta admisible y A^* recupera la ruta óptima $[7, 6, 4]$.

El experimento ilustra de forma concreta la afirmación teórica del enunciado: A^* devuelve el camino óptimo *si y solo si* la heurística satisface $h(n) \leq h^*(n)$. La práctica integradora Lab-MR 6 reutiliza A^* (con heurística euclídea consistente, garantizada al construir una matriz de costes compatible) como planificador global de la navegación autónoma.